

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
```

```
"""
U.S. Childcare Cost Analysis Project (2008-2018)
DSC640 - Data Presentation and Visualization
Student: Komal Shahid
Date: March 2, 2024
```

This script generates static visualizations for the childcare cost analysis:

1. Time series analysis of childcare costs
2. Geographic distribution (choropleth map)
3. Female labor force participation (geographic distribution)
4. Cost distribution analysis
5. Correlation heatmap
6. Cost trends analysis

```
"""
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import geopandas as gpd
from pathlib import Path
import os

# Set style for all plots
plt.style.use('seaborn-v0_8')

# Define color palettes
colors = sns.cubehelix_palette(8, start=.5, rot=-.75) # Updated color palette

# Set up global paths
script_dir = Path(__file__).parent
output_dir = script_dir / 'output'
output_dir.mkdir(exist_ok=True)
img_dir = output_dir / 'images'
img_dir.mkdir(exist_ok=True)

# Define state neighbors for missing value imputation
STATE_NEIGHBORS = {
    'IN': ['IL', 'KY', 'MI', 'OH'], # Indiana's neighbors
    'NM': ['AZ', 'CO', 'OK', 'TX'] # New Mexico's neighbors
}

def get_state_value(df, state, column, year):
    """
    Get value for a state by trying different strategies.

    Parameters:
    -----
    df : DataFrame
        The input dataframe containing the data
    state : str
        The state abbreviation
    column : str
        The column name for which to get the value
    year : int
        The year for which to get the value

    Returns:
    -----
    float
        The imputed value based on the best available strategy
    """
    state_data = df[df['State_Abbreviation'] == state]

    # Strategy 1: Try to get the exact value for this state and year
    exact_value = state_data[state_data['StudyYear'] == year][column].mean()
    if not pd.isna(exact_value):
        return exact_value

    # Strategy 2: Try to get value from the same state in different years
    state_mean = state_data[column].mean()
    if not pd.isna(state_mean):
        return state_mean

    # Strategy 3: Try to get value from neighboring states in the same year
    if state in STATE_NEIGHBORS:
        neighbor_data = df[
            (df['State_Abbreviation'].isin(STATE_NEIGHBORS[state])) &
            (df['StudyYear'] == year)
        ]
        neighbor_mean = neighbor_data[column].mean()
        if not pd.isna(neighbor_mean):
            return neighbor_mean
```

```

# Strategy 4: Use the global mean as a last resort
return df[column].mean()

def load_actual_data():
    """Load actual data from the National Database of Childcare Prices."""
    data_path = Path(__file__).parent / '../data/nationaldatabaseofchildcareprices.xlsx'
    print(f"\nLoading data from: {data_path}")

    # Load the data
    df = pd.read_excel(data_path)
    print(f"\nInitial data shape: {df.shape}")

    # Based on the technical guide, we'll keep only essential columns
    # Core location and time identifiers
    core_columns = ['State_Name', 'State_Abbreviation', 'County_Name', 'County_FIPS_Code', 'StudyYear']

    # Key demographic variables
    demographic_columns = ['TotalPop', 'H_Under6_BothWork', 'H_Under6_SingleM',
                           'H_6to17_BothWork', 'H_6to17_SingleM', 'MHI_2018']

    # Key childcare price metrics (keeping only the main provider types)
    # MC: Center-based care
    # MFCC: Family childcare homes
    childcare_columns = ['MCInfant', 'MCToddler', 'MCPreschool',
                        'MFCCInfant', 'MFCCToddler', 'MFCCPreschool']

    # Columns to keep
    columns_to_keep = core_columns + demographic_columns + childcare_columns

    # Check what percentage of data is missing in key columns
    print("\nMissing data in key childcare columns:")
    for col in childcare_columns:
        missing_pct = df[col].isna().mean() * 100
        print(f"{col}: {missing_pct:.1f}% missing")

    # Filter to keep only essential columns
    df_filtered = df[columns_to_keep].copy()
    print(f"\nFiltered to {len(columns_to_keep)} essential columns from {df.shape[1]} original columns")

    print("\nChecking data for IN and NM before any processing:")
    for state in ['IN', 'NM']:
        state_data = df_filtered[df_filtered['State_Abbreviation'] == state]
        if len(state_data) > 0:
            print(f"\n{state} raw data:")
            print(f"Number of rows: {len(state_data)}")
            print(f"Years covered: {sorted(state_data['StudyYear'].unique())}")
            print(f"\nSample row for {state}:")
            print(state_data[['StudyYear', 'State_Abbreviation', 'MCInfant', 'MCToddler', 'MCPreschool']].head(
1).to_string())
            print(f"\nValue counts for MCInfant:")
            print(state_data['MCInfant'].value_counts(dropna=False))

    print("\nHandling missing values...")
    # Create a copy of the dataframe to store the processed data
    processed_df = df_filtered.copy()

    # Handle missing values using the state-based approach for key childcare columns
    for col in childcare_columns:
        missing_mask = df_filtered[col].isna()
        if missing_mask.any():
            print(f"\nHandling {missing_mask.sum()} missing values in {col}")
            for idx in df_filtered[missing_mask].index:
                state = df_filtered.loc[idx, 'State_Abbreviation']
                year = df_filtered.loc[idx, 'StudyYear']
                value = get_state_value(df_filtered, state, col, year)
                processed_df.loc[idx, col] = value

    # Print sample of filled values for verification
    sample_states = ['IN', 'NM']
    for state in sample_states:
        state_data = processed_df[processed_df['State_Abbreviation'] == state]
        if len(state_data) > 0:
            print(f"\n{state} {col} sample filled values (first 5 rows):")
            print(state_data[['StudyYear', col]].head().to_string())

    # Calculate derived metrics
    print("\nCalculating derived metrics...")
    processed_df['Annual_Cost_Infant'] = processed_df['MCInfant'] * 12
    processed_df['Annual_Cost_Toddler'] = processed_df['MCToddler'] * 12
    processed_df['Annual_Cost_Preschool'] = processed_df['MCPreschool'] * 12
    processed_df['Cost_Burden'] = (processed_df['Annual_Cost_Infant'] / processed_df['MHI_2018']) * 100
    processed_df['Working_Parent_Ratio'] = (processed_df['H_Under6_BothWork'] / processed_df['TotalPop']) * 100

    # Add Urban/Rural classification based on population
    processed_df['Urban_Rural'] = processed_df['TotalPop'].apply(lambda x: 'Urban' if x > 500000 else 'Rural')

```

```

# Filter data for 2008-2018
processed_df = processed_df[(processed_df['StudyYear'] >= 2008) & (processed_df['StudyYear'] <= 2018)]
print(f"\nFiltered data to 2008-2018: {len(processed_df)} rows")

# Check processed data
print("\nChecking IN and NM data after processing:")
for state in ['IN', 'NM']:
    state_data = processed_df[processed_df['State_Abbreviation'] == state]
    if len(state_data) > 0:
        print(f"\n{state} processed data:")
        print(f"Number of rows: {len(state_data)}")
        print(f"Years covered: {sorted(state_data['StudyYear'].unique())}")
        print(f"\nSummary statistics for Annual_Cost_Infant:")
        print(state_data['Annual_Cost_Infant'].describe())
        print(f"\nSample processed rows:")
        print(state_data[['StudyYear', 'State_Abbreviation', 'Annual_Cost_Infant', 'MCInfant']].head().to_s
tring())

# Print information about data sources
print("\nData sources used for MCInfant:")
missing_count = df_filtered[df_filtered['State_Abbreviation'] == state]['MCInfant'].isna().sum()
if missing_count > 0:
    print(f"Values imputed from neighboring states: {STATE_NEIGHBORS[state]}")

return processed_df

def save_visualizations():
    """Save all static visualizations as PNG files."""
    print("Generating static visualizations...")

    # Load and process data
    df = load_actual_data()

    # Load geographic data
    shapefile_path = script_dir / 'data/cb_2018_us_state_20m/cb_2018_us_state_20m.shp'
    states_gdf = gpd.read_file(shapefile_path)

    # Generate all visualizations
    create_time_series(df)
    create_urban_rural_comparison(df)
    create_cost_distribution(df)
    create_state_costs_visualization(df)
    create_choropleth_maps(df, states_gdf)
    create_correlation_analysis(df)
    create_spiral_plot(df)

    print(f"\nAll static visualizations saved in: {img_dir}")

def create_time_series(df):
    """Create a time series visualization showing trends in childcare costs."""
    plt.figure(figsize=(12, 8), facecolor='#F0F0F8')

    # Calculate annual averages by year
    yearly_data = df.groupby('StudyYear')[['Annual_Cost_Infant', 'Annual_Cost_Toddler', 'Annual_Cost_Preschool']]
    .mean()

    # Create color palette using cubehelix
    palette = sns.cubehelix_palette(start=.5, rot=-.75, n_colors=3)

    # Plot the time series
    plt.plot(yearly_data.index, yearly_data['Annual_Cost_Infant'], marker='o', linewidth=2,
             color=palette[0], label='Infant Care')
    plt.plot(yearly_data.index, yearly_data['Annual_Cost_Toddler'], marker='s', linewidth=2,
             color=palette[1], label='Toddler Care')
    plt.plot(yearly_data.index, yearly_data['Annual_Cost_Preschool'], marker='^', linewidth=2,
             color=palette[2], label='Preschool')

    # Customize appearance
    plt.title('Trends in Average Annual Childcare Costs (2008-2018)',
             fontsize=16, fontweight='bold', pad=20)
    plt.xlabel('Year', fontsize=12, fontweight='bold')
    plt.ylabel('Average Annual Cost ($)', fontsize=12, fontweight='bold')
    plt.grid(True, linestyle='--', alpha=0.7)

    # Format y-axis with dollar signs and commas
    plt.gca().yaxis.set_major_formatter(plt.FuncFormatter(lambda x, p: f'${x:, .0f}'))

    # Add value labels on the lines
    for year in yearly_data.index:
        # Only label first, middle, and last points to avoid clutter
        if year in [yearly_data.index[0], yearly_data.index[len(yearly_data)//2], yearly_data.index[-1]]:
            for i, col in enumerate(['Annual_Cost_Infant', 'Annual_Cost_Toddler', 'Annual_Cost_Preschool']):
                plt.text(year, yearly_data[col][year] * 1.02,
                        f'${yearly_data[col][year]:, .0f}',
                        ha='center', fontsize=9)

```

```
plt.legend(loc='upper left', fontsize=10)
plt.tight_layout()
plt.savefig(img_dir / 'time_series.png', dpi=300, bbox_inches='tight', facecolor='#F0F0F8')
plt.close()
print("Saved: time_series.png")

def create_urban_rural_comparison(df):
    """Create a bar chart comparing childcare costs in urban vs. rural areas."""
    plt.figure(figsize=(12, 8), facecolor='#F0F0F8')

    # Group by urban/rural and calculate means
    grouped_data = df.groupby('Urban_Rural')[['Annual_Cost_Infant', 'Annual_Cost_Toddler', 'Annual_Cost_Preschool']].mean()

    # Create a bar chart
    bar_width = 0.25
    x = np.arange(len(grouped_data.index))

    # Create color palette using cubehelix
    palette = sns.cubehelix_palette(start=.5, rot=-.75, n_colors=3)

    # Create the bars
    plt.bar(x - bar_width, grouped_data['Annual_Cost_Infant'], width=bar_width,
            color=palette[0], label='Infant Care')
    plt.bar(x, grouped_data['Annual_Cost_Toddler'], width=bar_width,
            color=palette[1], label='Toddler Care')
    plt.bar(x + bar_width, grouped_data['Annual_Cost_Preschool'], width=bar_width,
            color=palette[2], label='Preschool')

    # Customize appearance
    plt.title('Urban vs. Rural Childcare Costs (2008-2018)',
              fontsize=16, fontweight='bold', pad=20)
    plt.xlabel('Area Type', fontsize=12, fontweight='bold')
    plt.ylabel('Average Annual Cost ($)', fontsize=12, fontweight='bold')
    plt.xticks(x, grouped_data.index, fontsize=10)
    plt.grid(True, linestyle='--', alpha=0.7, axis='y')

    # Format y-axis with dollar signs and commas
    plt.gca().yaxis.set_major_formatter(plt.FuncFormatter(lambda x, p: f'${x:, .0f}'))

    # Add value labels on the bars
    for i, area in enumerate(grouped_data.index):
        for j, col in enumerate(['Annual_Cost_Infant', 'Annual_Cost_Toddler', 'Annual_Cost_Preschool']):
            plt.text(i + (j-1)*bar_width, grouped_data[col][area] + 100,
                    f'${grouped_data[col][area]:, .0f}',
                    ha='center', fontsize=9)

    plt.legend(loc='upper right', fontsize=10)
    plt.tight_layout()
    plt.savefig(img_dir / 'urban_rural_comparison.png', dpi=300, bbox_inches='tight')
    plt.close()
    print("Saved: urban_rural_comparison.png")

def create_cost_distribution(df):
    """Create a violin plot showing the cost distribution across care types."""
    # Set up the figure with a nice background
    plt.figure(figsize=(12, 8), facecolor='white')

    # Set up care types
    care_types = ['MCInfant', 'MCToddler', 'MCPreschool']
    care_labels = ['Infant', 'Toddler', 'Preschool']

    # Prepare data for violin plot
    plot_data = pd.DataFrame()
    for ct, label in zip(care_types, care_labels):
        temp_df = pd.DataFrame({
            'Care Type': label,
            'Monthly Cost ($)': df[ct]
        })
        plot_data = pd.concat([plot_data, temp_df])

    # Create color palette using cubehelix
    palette = sns.cubehelix_palette(start=.5, rot=-.75, n_colors=3)

    # Create violin plot with seaborn - fix the warning by explicitly setting hue
    ax = sns.violinplot(data=plot_data, x='Care Type', y='Monthly Cost ($)',
                       hue='Care Type', legend=False,
                       palette=palette,
                       inner='box',
                       linewidth=1)

    plt.title('Distribution of Monthly Childcare Costs by Age Group (2008-2018)',
              pad=20, fontsize=16, fontweight='bold')
    plt.xlabel('Care Type', fontsize=12, fontweight='bold')
    plt.ylabel('Monthly Cost ($)', fontsize=12, fontweight='bold')
```

```
# Format y-axis with dollar signs
from matplotlib.ticker import FuncFormatter
ax.yaxis.set_major_formatter(FuncFormatter(lambda x, p: f'${x:,.0f}'))

# Add grid lines for better readability
plt.grid(axis='y', linestyle='--', alpha=0.7)

# Add summary statistics with background boxes for readability
for i, care_type in enumerate(care_types):
    mean_cost = df[care_type].mean()
    median_cost = df[care_type].median()

    # Create a background box for the statistics
    bbox_props = dict(boxstyle="round,pad=0.3", fc="white", ec="gray", alpha=0.8)

    plt.text(i, df[care_type].max() * 0.95,
             f'Mean: ${mean_cost:,.0f}\nMedian: ${median_cost:,.0f}',
             ha='center', va='top',
             fontsize=10,
             bbox=bbox_props)

# Remove top and right spines
sns.despine()

# Add note about data
plt.figtext(0.99, 0.01,
           "Note: Data from all states (2008-2018), including imputed values for missing data",
           ha='right', fontsize=8, style='italic')

# Save the figure
plt.tight_layout()
plt.savefig(img_dir / 'cost_distribution.png', dpi=300, bbox_inches='tight', facecolor='white')
plt.close()
print("Saved: cost_distribution.png")

def create_choropleth_maps(df, states_gdf):
    """Create choropleth maps for costs and labor force participation."""
    # Latest year data
    latest_year = df['StudyYear'].max()
    latest_data = df[df['StudyYear'] == latest_year]

    # Merge with geographic data
    states = states_gdf.merge(latest_data, left_on='STUSPS', right_on='State_Abbreviation', how='left')

    # Set up figure for cost map
    fig, ax = plt.subplots(figsize=(15, 10), facecolor='white')

    # Use cubehelix color palette
    cmap = sns.cubehelix_palette(start=.5, rot=-.75, as_cmap=True)

    # Set bounds to focus on continental US
    ax.set_xlim(-125, -66.5)
    ax.set_ylim(24, 50)

    # Create the choropleth
    states.plot(column='Annual_Cost_Infant',
               ax=ax,
               cmap=cmap,
               edgecolor='white',
               linewidth=0.5,
               legend=True,
               legend_kwds={
                   'label': "Annual Infant Childcare Cost ($)",
                   'orientation': "horizontal",
                   'shrink': 0.6,
                   'fraction': 0.035,
                   'pad': 0.01,
                   'fmt': "${x:,.0f}"
               },
               missing_kwds={'color': 'lightgray'})

    # Add state boundaries
    states_gdf.boundary.plot(ax=ax, color='black', linewidth=0.3, alpha=0.5)

    # Add state labels with text boxes for better visibility
    for idx, row in states.iterrows():
        # Skip Alaska and Hawaii for better focus on continental US
        if row['STUSPS'] not in ['AK', 'HI'] and not pd.isna(row['Annual_Cost_Infant']):
            centroid = row.geometry.centroid
            cost_value = row['Annual_Cost_Infant']

            # Calculate text color based on cost value
            text_color = 'white' if cost_value > states['Annual_Cost_Infant'].median() else 'black'

            # Add a small background box behind the text
```

```
box_color = 'black' if text_color == 'white' else 'white'
bbox_props = dict(boxstyle="round,pad=0.3", fc=box_color, ec=box_color, alpha=0.5)

# Add the annotation with the background box
ax.annotate(f"{row['STUSPS']}\n${cost_value:,.0f}",
            xy=(centroid.x, centroid.y),
            ha='center', va='center',
            fontsize=8,
            color=text_color,
            fontweight='bold',
            bbox=bbox_props)

# Customize the map
ax.set_title(f'Annual Infant Childcare Costs by State ({latest_year})',
            fontsize=18, fontweight='bold', pad=20)
ax.set_axis_off()

# Add note about data
note = "Note: Data for IN and NM derived from neighboring states"
plt.figtext(0.99, 0.01, note, ha='right', fontsize=8, style='italic')

# Save the map
plt.tight_layout()
plt.savefig(img_dir / 'cost_map.png', dpi=300, bbox_inches='tight', facecolor='white')
plt.close()
print("Saved: cost_map.png")

# Working parent ratio map - Second Map
fig, ax = plt.subplots(figsize=(15, 10), facecolor='white')

# Use a different cubehelix palette for the second map
cmap2 = sns.cubehelix_palette(start=2.8, rot=-.5, as_cmap=True)

# Set bounds to focus on continental US
ax.set_xlim(-125, -66.5)
ax.set_ylim(24, 50)

# Create the choropleth
states.plot(column='Working_Parent_Ratio',
            ax=ax,
            cmap=cmap2,
            edgecolor='white',
            linewidth=0.5,
            legend=True,
            legend_kwds={
                'label': "% of Households with Children Under 6,\nBoth Parents Working",
                'orientation': "horizontal",
                'shrink': 0.6,
                'fraction': 0.035,
                'pad': 0.01,
                'fmt': "{x:.1f}%"
            },
            missing_kwds={'color': 'lightgray'})

# Add state boundaries
states_gdf.boundary.plot(ax=ax, color='black', linewidth=0.3, alpha=0.5)

# Add state labels with text boxes for better visibility
for idx, row in states.iterrows():
    # Skip Alaska and Hawaii for better focus on continental US
    if row['STUSPS'] not in ['AK', 'HI'] and not pd.isna(row['Working_Parent_Ratio']):
        centroid = row.geometry.centroid
        ratio_value = row['Working_Parent_Ratio']

        # Calculate text color based on ratio value
        text_color = 'white' if ratio_value > states['Working_Parent_Ratio'].median() else 'black'

        # Add a small background box behind the text
        box_color = 'black' if text_color == 'white' else 'white'
        bbox_props = dict(boxstyle="round,pad=0.3", fc=box_color, ec=box_color, alpha=0.5)

        # Add the annotation with the background box
        ax.annotate(f"{row['STUSPS']}\n{ratio_value:,.1f}%",
                    xy=(centroid.x, centroid.y),
                    ha='center', va='center',
                    fontsize=8,
                    color=text_color,
                    fontweight='bold',
                    bbox=bbox_props)

# Customize the map
ax.set_title(f'Working Parents with Young Children by State ({latest_year})',
            fontsize=18, fontweight='bold', pad=20)
ax.set_axis_off()

# Save the map
```

```
plt.tight_layout()
plt.savefig(img_dir / 'labor_force_map.png', dpi=300, bbox_inches='tight', facecolor='white')
plt.close()
print("Saved: labor_force_map.png")

def create_correlation_analysis(df):
    """Create a correlation heatmap for key metrics."""
    plt.figure(figsize=(12, 10), facecolor='#F0F0F8')

    # Create correlation matrix with just the columns we need
    corr_vars = [
        'Annual_Cost_Infant', 'Annual_Cost_Toddler', 'Annual_Cost_Preschool',
        'Cost_Burden', 'Working_Parent_Ratio', 'TotalPop', 'MHI_2018'
    ]

    # Filter to the latest year data
    latest_year = df['StudyYear'].max()
    latest_data = df[df['StudyYear'] == latest_year]

    # Create the correlation matrix
    corr_matrix = latest_data[corr_vars].corr()

    # Generate a mask for the upper triangle
    mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

    # Create heatmap with cubehelix palette
    cmap = sns.cubehelix_palette(start=.5, rot=-.75, as_cmap=True)
    sns.heatmap(corr_matrix, mask=mask, cmap=cmap, vmax=1, vmin=-1, center=0,
                square=True, linewidths=.5, cbar_kws={"shrink": .8},
                annot=True, fmt=".2f")

    plt.title('Correlation Between Childcare Costs and Socioeconomic Factors',
              fontsize=16, fontweight='bold', pad=20)

    plt.tight_layout()
    plt.savefig(img_dir / 'correlation.png', dpi=300, bbox_inches='tight')
    plt.close()
    print("Saved: correlation.png")

def create_spiral_plot(df):
    """Create a spiral plot showing childcare costs over time for different care types."""
    plt.figure(figsize=(12, 12), facecolor='#F0F0F8')

    # Set up basic plot
    ax = plt.subplot(111, projection='polar')
    ax.set_facecolor('#F0F0F8')

    # Data preparation
    years = sorted(df['StudyYear'].unique())
    theta = np.linspace(0, 2*np.pi, len(years), endpoint=False)

    # Calculate average costs by year for different care types
    yearly_means = df.groupby('StudyYear').agg({
        'MCInfant': 'mean',
        'MCToddler': 'mean',
        'MCPreschool': 'mean'
    }).reset_index()

    # Order by year
    yearly_means = yearly_means.sort_values('StudyYear')

    # Create color palette using cubehelix
    palette = sns.cubehelix_palette(start=.5, rot=-.75, n_colors=3)

    # Plot each care type
    care_types = ['MCInfant', 'MCToddler', 'MCPreschool']
    care_type_labels = ['Infant', 'Toddler', 'Preschool']

    # Find max value for radius normalization
    max_value = yearly_means[care_types].max().max()

    for i, (care_type, label) in enumerate(zip(care_types, care_type_labels)):
        values = yearly_means[care_type]
        # Normalize values to [0.2, 1.0] range
        r = 0.2 + 0.8 * (values / max_value)

        # Plot line
        ax.plot(theta, r, '-', color=palette[i], linewidth=2, label=label)

        # Add points for each year
        ax.scatter(theta, r, s=100, color=palette[i], alpha=0.8, edgecolor='w')

        # Add value labels
        for j, (year, value) in enumerate(zip(years, values)):
            # Only label first and last year
            if j == 0 or j == len(years) - 1:
```

```

        angle = theta[j]
        radius = r[j]

        # Adjust text position slightly outside the point
        text_r = radius * 1.1

        # Use polar to cartesian conversion to place text
        x = text_r * np.cos(angle)
        y = text_r * np.sin(angle)

        # Adjust text alignment based on angle
        ha = 'left' if x > 0 else 'right'
        va = 'bottom' if y > 0 else 'top'

        plt.annotate(f'${value:.0f}',
                    (angle, radius),
                    xytext=(x * 1.1, y * 1.1),
                    textcoords='data',
                    ha=ha, va=va,
                    fontsize=10,
                    fontweight='bold',
                    color=palette[i])

    # Set the y-ticks (radial ticks)
    yticks = np.linspace(0.2, 1.0, 5)
    ytick_labels = [f'${int(tick * max_value):.0f}' for tick in yticks]
    ax.set_yticks(yticks)
    ax.set_yticklabels(ytick_labels)

    # Set the x-ticks (angular ticks)
    ax.set_xticks(theta)
    ax.set_xticklabels(years, fontsize=10)

    # Add title and legend
    plt.title('Monthly Childcare Costs by Age Group (2008-2018)',
             fontsize=16, fontweight='bold', pad=20)
    plt.legend(loc='upper right', fontsize=12)

    plt.tight_layout()
    plt.savefig(img_dir / 'spiral_plot.png', dpi=300, bbox_inches='tight')
    plt.close()
    print("Saved: spiral_plot.png")

def create_state_costs_visualization(df):
    """Create a horizontal bar chart showing average childcare costs by state with standard deviation."""
    # Create figure with specific size and layout
    fig, ax = plt.subplots(figsize=(16, 20))

    # Calculate mean and standard deviation for each state
    state_stats = df.groupby('State_Abbreviation')['Annual_Cost_Infant'].agg(['mean', 'std']).reset_index()

    # Calculate standard deviation as percentage of mean
    state_stats['std_pct'] = (state_stats['std'] / state_stats['mean']) * 100

    # Sort by mean cost
    state_stats = state_stats.sort_values('mean', ascending=True)

    # Ensure standard deviations are valid
    state_stats['std_pct'] = state_stats['std_pct'].fillna(0)

    # Create color mapping using cubehelix
    norm = plt.Normalize(state_stats['mean'].min(), state_stats['mean'].max())
    cmap = sns.cubehelix_palette(start=.5, rot=-.75, as_cmap=True)
    colors = [cmap(norm(x)) for x in state_stats['mean']]

    # Create horizontal bars with error bars
    y_pos = np.arange(len(state_stats))
    bars = ax.barh(y_pos, state_stats['mean'],
                  xerr=state_stats['std'],
                  color=colors,
                  capsize=5, alpha=0.7)

    # Customize the plot
    ax.set_yticks(y_pos)
    ax.set_yticklabels(state_stats['State_Abbreviation'], fontsize=12)
    ax.set_xlabel('Average Annual Cost ($)', fontsize=14, fontweight='bold')
    ax.set_ylabel('State', fontsize=14, fontweight='bold')

    # Set x-axis to start from 0
    ax.set_xlim(0, state_stats['mean'].max() * 1.2) # 20% padding for labels

    # Format x-axis with comma separator and no decimals
    ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, p: f'${x:,.0f}'))

    title = 'Average Childcare Costs by State (2008-2018)\nwith Standard Deviation'
    ax.set_title(title, fontsize=16, fontweight='bold', pad=20)

```

```
# Add colorbar
sm = plt.cm.ScalarMappable(cmap=cmap, norm=norm)
sm.set_array([])
cbar = plt.colorbar(sm, ax=ax)
cbar.set_label('Average Annual Cost ($)', fontsize=12, fontweight='bold')

# Add grid for better readability
ax.grid(True, axis='x', alpha=0.3)

# Add value labels with mean and standard deviation percentage
for i, (mean, std_pct) in enumerate(zip(state_stats['mean'], state_stats['std_pct'])):
    ax.text(mean + 100, i, f'${mean:,.0f} (Â±{std_pct:.1f}%)',
            va='center', fontsize=9)

# Add note about data sources for IN and NM
note = "Note: Data for IN and NM includes values derived from neighboring states"
plt.figtext(0.99, 0.01, note, ha='right', fontsize=8, style='italic')

# Adjust layout and save
plt.tight_layout()
plt.savefig(img_dir / 'state_costs.png', dpi=300, bbox_inches='tight')
plt.close()
print("Saved: state_costs.png")

if __name__ == "__main__":
    save_visualizations()
```